ChatGPT

Application Purpose & User Roles

A modern salon app must cater to **five distinct user roles**: Super Admin (platform owner), Salon Owner (branch manager), Staff (stylists/receptionists), Vendor (e.g. product supplier or salon-as-vendor), and Customer (end client). In general, the app's purpose is to **automate salon workflows**—scheduling, payments, inventory, staff management, marketing and analytics—in a multi-location (multi-tenant) setting [1] [2]. For each role, the requirements are:

- **Super Admin:** Oversees all salons/branches. Needs **global visibility and control**. For example, a super-admin "can access and manage all branches and staff" and **create or assign branch admins** [2]. They set platform-wide policies (pricing, roles, security) and view consolidated reports across all salons (bookings, revenue, staff performance) [2] [3]. Every authorization check must be *tenant-aware*: e.g. "is this user an admin *for this salon*?" [3]. In practice the super-admin portal includes user/RBAC management, global analytics, and multi-tenant configuration (no cross-tenant data leakage) [4] [5].

- **Salon Owner (Branch Admin):** Manages one salon location. Needs a **rich dashboard** and permission to run daily operations. Typical features include managing services offered, pricing, staff schedules, client lists, inventory ordering, and local financials. The owner should "manage services, staff schedules, appointments, payments, promotions, customer records, and even track sales and inventory" [6]. They can add/edit staff users, set roles for them (e.g. manager vs. cashier), and oversee branch-specific reports [7] [8]. Owners use analytics to make data-driven decisions (peak hours, top services, staffing needs) [8]. In a multi-branch chain, each owner only sees their own branch data [7] [4].

- **Staff:** Salon employees (stylists, receptionists, etc.). Need **scheduling tools and booking controls** for their own appointments. The app must let staff log in, view/edit their upcoming appointments, manage client check-in, and update booking status. In practice, staff "can log in to check their hours, view upcoming appointments, and even request time off" [9]. Their permissions are limited to their own schedule and clients. Some salons also track staff productivity or commission (appointments completed, sales). Staff UIs typically include personal schedule views and client history lookup.

- **Vendor:** This role can be interpreted two ways. In a **multi-vendor marketplace** model, "vendor" often means a salon branch owner registered on the platform. In that case the vendor's needs are identical to the Salon Owner role. More commonly in a salon context, "vendor" means **product supplier**. Such users would log in to an **inventory management portal**: they receive purchase orders and update stock. Many salon systems incorporate vendor/inventory management as a core module [10] [11]. For example, Zenoti's platform unifies salon operations "from commissions and payouts to inventory and *vendor management*" [10]. Thus a vendor user might handle product listings, confirm shipments, and manage invoicing.

- **Customer (End User):** Schedules services. The app must provide a **client-facing booking interface**. Customers create profiles, browse the **service catalog** (list of treatments with images, descriptions,

duration, price), and pick a time slot with a staff member. As one guide notes, clients should be able to "browse services, schedule appointments, view prices, [and] make payments" – all conveniently via the mobile app [1]. They receive confirmations and reminders. Post-visit, they may view appointment history, leave reviews, or redeem loyalty rewards.

Each role has distinct data flows. For example, a **booking use case**: a Customer books a haircut (date, time, stylist) → Owner/Staff approves and updates calendars → Customer pays via integrated POS → Staff provides service → System updates client history and applies any loyalty points. An **inventory use case**: when services are booked or products sold at the POS, inventory levels are adjusted, triggering automatic reorder alerts or vendor purchase orders [11]. (Mapping these flows in detail is critical in your architecture.)

Salon Workflows & Features

Drawing on industry best-practices and top salon software, the app should unify **all core salon workflows** into one platform [1] [9]. Key features (with examples from existing solutions) include:

- **Appointment Scheduling:** A robust booking calendar is essential. Clients must see available slots and book or cancel at will [12]. Salon owners schedule staff shifts and block off breaks. Staff and owners receive notifications (and can send reminders) to minimize no-shows.
- **Service Catalog:** A browsable list of all treatments with prices, descriptions, durations and optional media. Customers browse this catalog in-app before booking [12].
- **Client Management (CRM):** Store customer profiles with contact info, visit history, preferences and payment methods. This enables personalized service (e.g. remembering a client's favorite hair color) [8] [6]. CRM features also drive marketing (see below).
- **Point of Sale (POS) & Payments:** Integrated checkout lets customers pay in-app or at the salon. Support for major payment gateways and tips is expected [13]. The POS should automatically record sales, issue receipts, and track revenue by service and staff. Software like BookingBee cites POS integration as critical for reducing payment errors and calculating commissions [14].
- **Inventory Management:** Track products (shampoos, dyes, retail items) used in services. The system should log stock levels, set reorder points, and even auto-generate purchase orders to suppliers [11]. For example, most salon apps can "track stock levels, set reorder points, and place orders directly with suppliers when you're running low" [11]. This prevents out-of-stock issues and ties inventory costs to services.
- **Staff Scheduling & Payroll:** Easily assign staff to shifts and appointments. Staff can request time off or swap shifts. Track hours worked (for payroll) and performance (appointments served). BookingBee notes that integrated scheduling "cuts down on miscommunications" and helps track each stylist's revenue, aiding commission payouts [15].
- **Marketing & Loyalty:** Build client retention with promotions, memberships, and loyalty points. The app should send automated reminders, birthday/anniversary offers, and push notifications. Many salon solutions include loyalty programs (points, coupons) and email/SMS marketing to keep clients returning [16].
- **Reviews & Social:** Allow customers to rate services and submit feedback or photos. Social proof is important for business growth.
- **Reporting & Analytics:** Real-time dashboards for owners. Track daily/weekly sales, service popularity, client retention, staff productivity, etc [8] [17]. Actionable reports (e.g. best-selling

services, no-show rates, inventory usage) help optimize operations and inventory decisions. Good platforms enable exporting reports to PDF/Excel [17].

These features cover the full salon lifecycle. As one industry guide says, a salon app "unifies appointment scheduling, inventory, staff coordination, payments, marketing, [and] customer retention… bringing multiple currency benefits" [18]. In short, clients enjoy 24/7 booking and convenience, while the salon gains efficiency, data-driven insights and scalability [19] [20].

Security, Compliance & Multi-Tenancy

Given sensitive client data and payments, security is paramount. **Role-based access control (RBAC)** and data isolation are required by design. For example, scribd's salon system blueprint mandates "role-based permissions with encrypted credentials" and **isolated data for each branch** [5]. In a multi-tenant SaaS, every authorization must be *tenant-scoped*: instead of a global "isAdmin?", the check must be "is user an admin **for this salon tenant**?" [3]. A good RBAC model ensures "no cross-tenant reads or writes" and that roles/permissions are scoped to each salon [4]. In practice, each API call and screen should filter by the user's salon ID and verify their role (owner vs staff vs vendor vs customer).

Other security best-practices include: use **HTTPS/TLS** for all network communication; store tokens securely (e.g. in Keychain/Keystore, not AsyncStorage); encrypt sensitive data at rest; implement multi-factor authentication for administrative users [21]; and pin certificates if needed. Payment handling must be PCI-DSS compliant (or use a PCI-compliant gateway) [21]. If operating in regions with data privacy laws, the app should comply with GDPR/CCPA for client data protection [21]. The salon app should also have audit logging (record key actions by each role) and regular backups [5]. In summary, enforce the least-privilege access for each role and treat each salon's data as a separate "vault."

Phased Development Roadmap

Building an enterprise-grade app is complex; a phased roadmap ensures steady progress. A recommended plan might be:

1. **Phase 1 – Foundations & Core Functions:** Set up project structure and authentication (JWT with refresh tokens). Implement RBAC (super-admin, owner, staff, vendor, customer) and navigation flows per role. Build **Customer booking** (service catalog, appointment calendar, online payment) and **Owner dashboard** (view bookings, manage services). Implement basic **Staff scheduling** (staff can view their appointments and profile). This covers the minimal viable product (MVP) for the main user journeys.

2. **Phase 2 – Salon Operations:** Add **Inventory & POS** modules. Owners can track products and sell them via the POS; integrate supplier/vendor workflows. Complete **Staff management** (owner can add staff, assign shifts, payroll integration). Improve UX/UI (themes, responsive layouts) and add real-time features (e.g. live status updates).

3. **Phase 3 – Client Engagement:** Build marketing features (promotions, loyalty programs, push notifications), **Reviews/Feedback** system, and user profiles refinement. Polish the UI with

animations (see below) and advanced components. Incorporate multi-location support if expanding beyond one salon (tenants management).

4. **Phase 4 – Analytics & Scalability:** Deploy comprehensive reporting dashboards and analytics (sales, occupancy, staff stats). Optimize performance (e.g. enable offline caching, use Hermes engine). Conduct security audits and finalize compliance documentation. Prepare for CI/CD, crash reporting, and large-scale deployment.

Throughout, iterative testing and stakeholder feedback are crucial. This roadmap ensures owners get a functional app early (booking and staff management), then gradually unlocks business-driving features (inventory, marketing, analytics) [20] [17].

Boilerplates & Implementation Strategy

Given the scope, **starting with a solid codebase** can save time. Mature React Native boilerplates bundle best practices and common libraries (navigation, theming, state management, etc.). For example, MindInventory's React Native Boilerplate "contains all the basic packages, common components and prebuilt code architecture" – it's a turnkey foundation to reduce setup overhead [22]. Similarly, Infinite Red's **Ignite** boilerplate (used since 2016) is a battle-tested starter kit; Ignite's team notes it **"saves [developers] two to four weeks of time on average"** at a project's start [23]. These kits include TypeScript support, React Navigation, Redux (or other state libs), i18n, and much of the infrastructure you'll need.

However, boilerplates often include features you may not need, so evaluate them carefully. Another approach is to **build a custom scaffolding via CLI tools (e.g. React Native CLI + template)**, defining folder structure and libraries yourself. In either case, maintain a clear modular architecture (feature-based folders, shared components, API services) as outlined above. In practice, many teams combine both approaches: they use a boilerplate for project wiring and then follow a **detailed roadmap** to implement only the required features.

Ultimately, whether you "start from scratch" or "start from boilerplate," the key is disciplined planning: define your roles & features, set up RBAC early, and iterate by priority. Use the boilerplate to avoid reinventing authentication and navigation code, but tailor it to your salon use cases. Whichever path you choose, ensure your app remains maintainable, scalable, and secure as you advance beyond the boilerplate's basics [22] [23].

Sources: Industry guides and platform documentation for salon software and RBAC informed this analysis [1] [2] [9] [10] [21] [3] [22] [23]. These highlight typical salon features, user needs, and security/compliance requirements. The cited boilerplates demonstrate proven architectural patterns to jumpstart a React Native project.

---

[1] [6] [8] [12] [13] [18] [19] [20] Beauty Salon App Development Guide for Salon Owners
https://salon.v1tl.com/blog/beauty-salon-app-creation-guide

[2] [5] [7] Centralized Salon Management System | PDF | Mobile App | Databases
https://www.scribd.com/document/966475310/Salon-Management-Copy

[3] [4] How to design an RBAC model for multi-tenant SaaS — WorkOS
https://workos.com/blog/how-to-design-multi-tenant-rbac-saas

[9] [11] [14] [15] [16] [17] Guide to Hair Salon Software for Salon Owners and Managers - Booking Bee
https://bookingbee.ai/guide-to-hair-salon-software-for-salon-owners-and-managers/

[10] Salon Management Software & Appointment Book | Zenoti
https://www.zenoti.com/salon-management-software

[21] How to Secure a Salon App? Best Practices for Data Safety
https://www.jploft.com/blog/how-to-secure-a-salon-app

[22] GitHub - Mindinventory/react-native-boilerplate: The Boilerplate contains all the basic packages, common components and, prebuilt code architecture. It will save developer's project setup time. supporting both expo and react-native CLI. · GitHub
https://github.com/Mindinventory/react-native-boilerplate

[23] GitHub - infinitered/ignite: Infinite Red's battle-tested React Native project boilerplate, along with a CLI, component/model generators, and more! 9 years of continuous development and counting. · GitHub
https://github.com/infinitered/ignite